Project – High Level Design

on

IaC Provisioning for Non-Profit Support System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Submitted by:

Sr no	Student Name	Enrolment Number
1.	Ananya Subramanya Rao	EN22CS301120
2.	Anirudh Vyas	EN22CS301131
3.	Anushka Kashyap	EN22CS301180
4.	Atharv Chaturvedi	EN22CS301228
5.	Harshit Panchal	EN23CS3L1010

Group Name: Group 03D2

Project Number: DO-15

Industry Mentor Name: Kunal Sharma Sir

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2026

Table of Contents

1. **Introduction**
   1.1. Scope of the document
   1.2. Intended Audience
   1.3. System overview
2. **System Design**
   2.1. Application Design
   2.2. Process Flow
   2.3. Information Flow
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. **Data Design**
4. **Interfaces**
5. **Infrastructure & Deployment**
6. **State and Session Management**
7. **Caching**
8. **Non-Functional Requirements**
9. **References**

1. Introduction

In today's technology-driven world, non-profit organizations struggle with fragmented tooling — one system for communication triage and another for staff training. Managing these separately creates operational inefficiency, data silos, and increased infrastructure complexity. SupportHub addresses this challenge by unifying two AI-powered microservices under a single, production-grade DevOps infrastructure.

SupportHub is a production-grade, AI-Powered microservices-based DevOps platform that automates the provisioning, containerization, and deployment of two AI-powered non-profit tools — TriageAgent (an AI communication triage system) and QuizBot (an AI-driven staff training platform) — as a unified microservices platform. The platform uses Infrastructure as Code (IaC) tools, container orchestration, and CI/CD pipelines to eliminate all manual setup and enable repeatable, one-command deployment of the entire system. Built using Docker for containerization, managed through an Nginx as an API gateway, GitHub Actions for CI/CD automation, Terraform for infrastructure provisioning, and Kubernetes for container orchestration and deployed on AWS EC2 — demonstrating the complete DevOps lifecycle in one cohesive system.

Problem Statement

Create a production-grade reactive triage agent for Non-Profit communications, utilizing Large Language Models to automate initial response workflows. The system performs real-time classification of incoming text to determine urgency and intent, applies Named Entity Recognition (NER) to extract critical data points such as IDs, dates, and generates contextually accurate draft responses using an LLM API to accelerate resolution times.

Project Objectives

- **Microservices Architecture:** Deploy TriageAgent and QuizBot as independent, containerized microservices under a single Nginx API gateway.
- **Full Containerization**: Package all 5 services as Docker containers connected through a shared supporthub-network Docker bridge network.
- **CI/CD Pipeline**: GitHub Actions builds all 5 Docker images, runs tests, builds images and pushes to Docker Hub, and deploys on every push to main.
- **Infrastructure as Code**: Provision the entire platform using Terraform, it scripts provision and deploy the entire platform with a single command.
- **Kubernetes Orchestration**: K8s manages container lifecycle with Deployments, Services, and Ingress for auto-restart, health checks, and horizontal scaling.
- **AWS Deployment**: EC2 instance (t3.small, Ubuntu 24.04) hosts the live platform accessible at http://13.206.119.123:8080

1.1 Scope of the document

This High-Level Design (HLD) document describes the complete architectural design, DevOps infrastructure, and system structure of the SupportHub platform — an AI-Powered Microservices Platform for Non-Profit Support Operations. The document covers:

- Overall microservices architecture and container interaction design
- Docker implementation strategy — Dockerfiles, Compose, multi-stage builds, Nginx routing
- CI/CD pipeline — GitHub Actions 3-job automated workflow (test → build → push)
- Infrastructure as Code — Terraform provisioning with automated scripts
- Kubernetes orchestration — Deployments, Services, Ingress, health probes
- AWS EC2 deployment — Cloud infrastructure, security groups, live deployment
- Data design — Firebase Firestore schema, SQLite, ChromaDB vector storage
- Security, session management, caching, and non-functional requirements

This document does not cover low-level implementation details such as individual function logic or UI component internals — those are addressed in the LLD document.

1.2 Intended Audience

Audience	Relevance
Project Evaluators	Assess technical depth and correctness of the DevOps implementation
Team Members (Group 03D2)	Reference guide for every design decision made during the project
DevOps Engineers / Developers	Maintain Docker containers, Terraform scripts, GitHub Actions workflows, and K8s configs
Non-Profit Organization Staff	End users who benefit from reliable, always-on services maintained by the automated pipeline

1.3 System overview

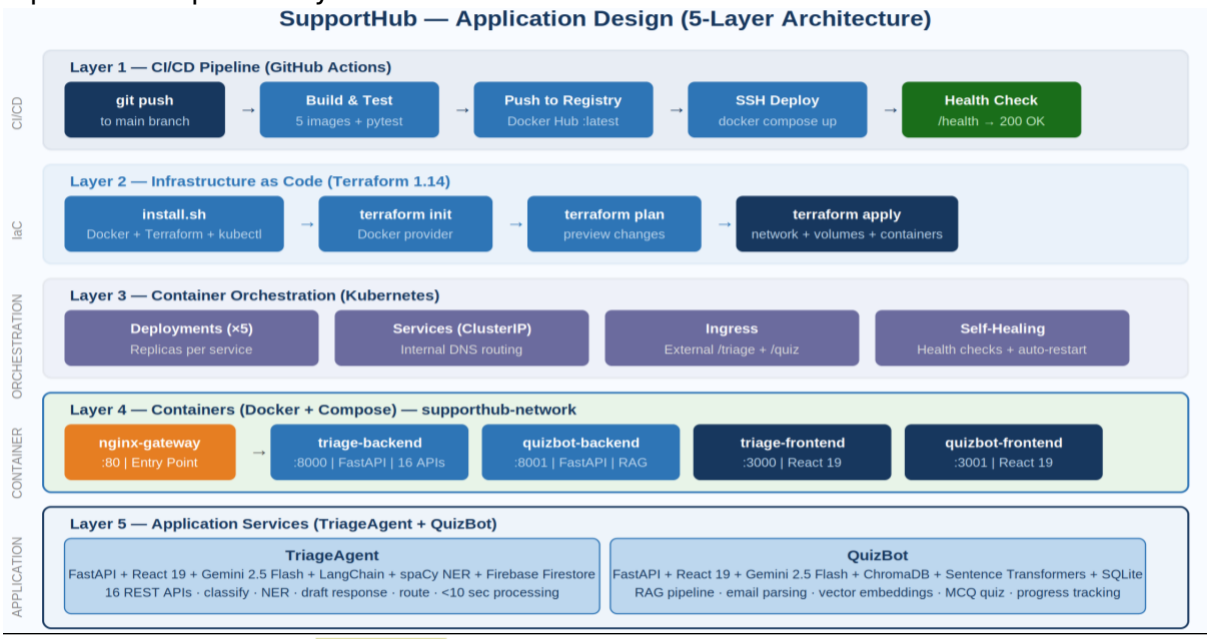
SupportHub deploys 5 Docker containers simultaneously on a shared bridge network called supporthub-network. All traffic enters through Nginx on port 80, which routes requests to the appropriate microservice. The platform processes the full DevOps lifecycle — from a developer git push to containers running live on an AWS EC2 server, with Kubernetes providing production-grade orchestration.

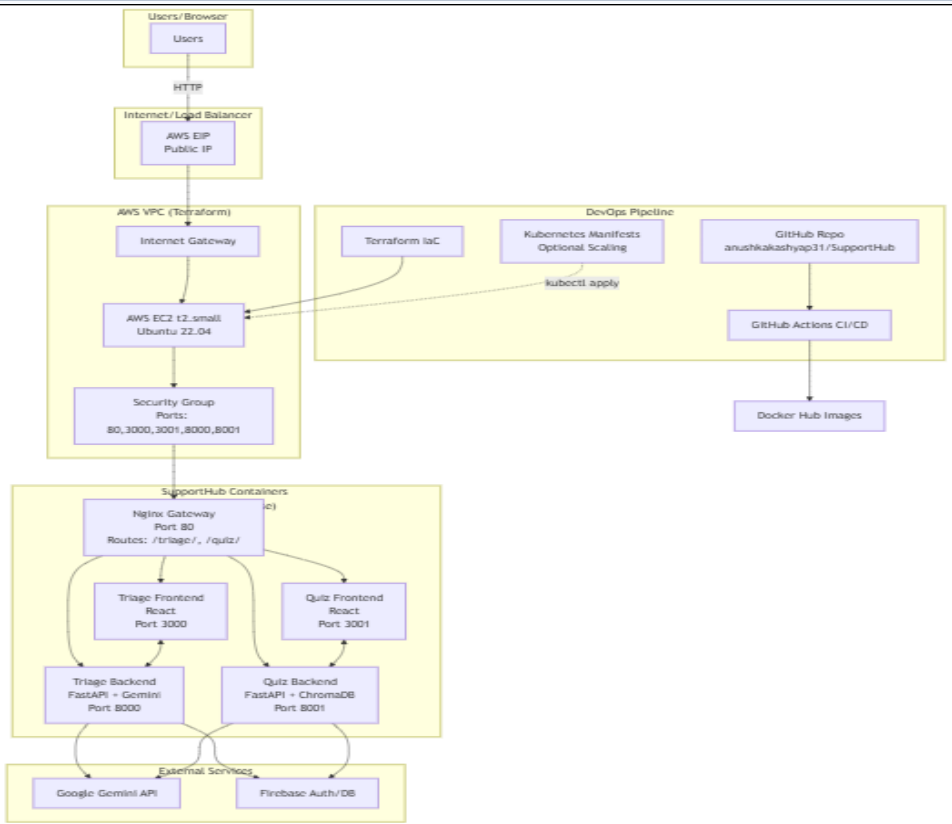
#	Component	Technology	Port	AWS Status
1	Nginx API Gateway	nginx:alpine + Custom Config	:8080	Live on EC2
2	TriageAgent Backend	Python 3.11 + FastAPI + LangChain + spaCy	:8000	Live on EC2
3	TriageAgent Frontend	React 19 + Vite 7 → nginx:alpine	:3000	Live on EC2
4	QuizBot Backend	Python 3.11 + FastAPI + ChromaDB + Transformers	:8001	Live on EC2
5	QuizBot Frontend	React 19 + Vite 7 → nginx:alpine	:3001	Live on EC2

2. System Design

2.1 Application Design

SupportHub follows a 5-Layer Containerized Microservices Architecture — two independent AI-powered applications (TriageAgent and QuizBot) deployed as separate containerized services under a unified API gateway and shared DevOps infrastructure, with CI/CD automation, IaC provisioning, Kubernetes orchestration, and an Nginx API gateway. Each layer has a single responsibility and clean interfaces — no layer bypasses another. This enforces separation of concerns and allows each layer to be scaled or replaced independently.





2.2 Process Flow

SupportHub has two distinct process flows. The CI/CD Pipeline Flow triggers automatically on every code push. The IaC Provisioning Flow runs once to set up fresh infrastructure on any new machine.

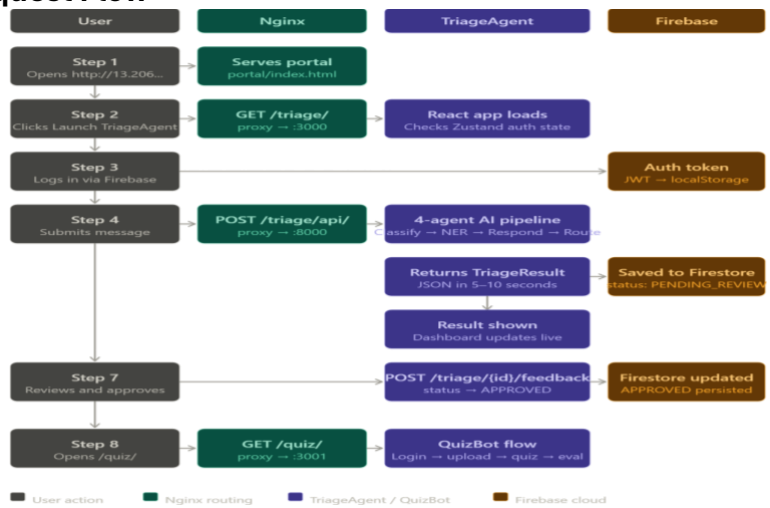
2.2.1 CI/CD Pipeline Process Flow (GitHub Actions)

Step	Job	What Happens	Duration
1	Trigger	Developer pushes code to main branch — ci-cd.yml workflow activates instantly	< 1s
2	Run Tests (Job 1)	Checkout code, setup Python 3.11, install dependencies, run py_compile syntax check on both backends	1m 12s
3	Build & Push (Job 2)	Login to Docker Hub using GitHub Secrets, build all 5 images with Docker Buildx, push to anushkakashyap31/ registry	5m 28s
4	Deploy (Job 3)	Validate docker-compose.yml config; in production would SSH and run docker compose up -d	7s
5	Complete	All 3 jobs GREEN — Run #7 total 6m 47s — 5 images live on Docker Hub	—

2.2.2 IaC Provisioning Flow (Terraform)

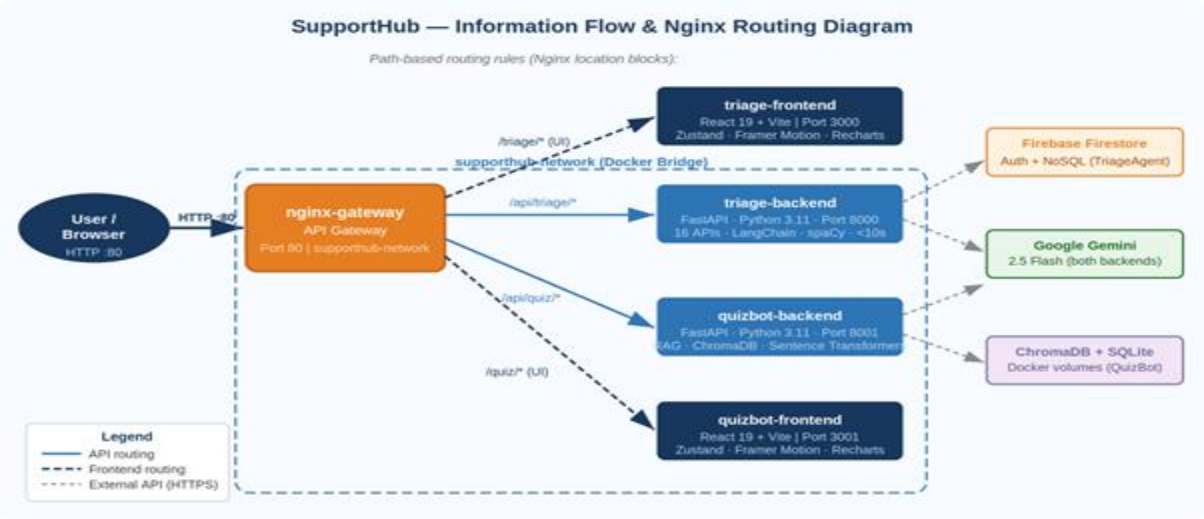
Step	Command	What Happens
1	./scripts/install.sh	Auto-installs Docker, Docker Compose, Terraform 1.14, and kubectl on any fresh Linux machine
2	terraform init	Initializes Terraform workspace, downloads hashicorp/aws provider
3	terraform plan	Previews all AWS resources to be created without making any changes
4	terraform apply	Creates VPC, subnet, internet gateway, security group, EC2 (t3.small), Elastic IP, installs K3s via user_data
5	docker compose up -d	Pulls latest images from Docker Hub, starts all 5 containers in detached mode
6	Platform live	http://13.206.119.123 — SupportHub accessible on the internet

2.2.3 User Request Flow



2.3 Information Flow

All user requests enter SupportHub through the Nginx API gateway on port 80. Nginx uses **upstream blocks and location-based rewrite rules** to route traffic. The SupportHub landing page (portal/index.html) is served at the root. Backends never receive external paths directly — Nginx rewrites and proxies them internally.



Here is the complete data transformation pipeline:

SN	Source	Destination	Format	Protocol
1	User Browser	Nginx API Gateway :80	HTTP Request	HTTP
2	Nginx Gateway	TriageAgent Frontend :3000 or QuizBot Frontend :3001	Proxied Request	Internal Network
3	React Frontend	FastAPI Backend via axios	JSON + Bearer Token	HTTPS REST
4	FastAPI Backend	AI Pipeline (Orchestrator)	Pydantic Model	Python Function
5	FirebaseService	Firebase Firestore / Auth	JSON Document	Firebase SDK
6	FastAPI	React Frontend	JSON Response	HTTP 200

Deployment Flow: The complete deployment flow from code commit to production follows a fully automated pipeline with minimal manual intervention required only for secret management.

```
DEVELOPER LAPTOP                        AWS CLOUD (ap-south-1)
|                                       |
| 1. git push main                      |
|         |                             |
|         v                             |
|   GitHub Actions                      |
|   [test->build->push]                 |
|         |                             |
|         v                             |
|   Docker Hub Registry                 |
|   5 images pushed                     |
|                                       |
| 2. terraform apply                    |
|         |                             |
|         +------------------------->|
|                                       | Creates VPC, EC2
|                                       | Elastic IP assigned
|                                       |
| 3. scp firebase-credentials.json      |
|         +------------------------->|
|                                       |
| 4. ssh ubuntu@EC2-IP                  |
|    nano .env (secrets)                |
|    docker compose up -d               |
|                                       | 5 containers running
|                                       |
|                                       | 5. K3s + kubectl apply -f k8s/
|                                       |    K8s pods running
|                                       |
INTERNET USER                           |
http://EC2-IP  <--------------------+
SupportHub LIVE!
```

2.4 Key Design Considerations

No.	Design Consideration	Description	Why This Choice Was Made
1	**Microservices Architecture**	TriageAgent and QuizBot deployed as independent services, each with own containers, communicating through Nginx.	Each service scales and fails independently. Two distinct AI pipelines stay isolated. Follows real-world industry patterns (Netflix, Zomato, Swiggy).
2	**Multi-Stage Docker Builds**	Frontend images use node:18-alpine to build React apps, then nginx:alpine to serve compiled static files. Each frontend also has its own nginx.conf.	Reduces final image size by 60–70%. Production images contain only compiled static files — no Node.js, no source code. Faster pulls and smaller attack surface.
3	**GitHub Actions over Jenkins**	CI/CD pipeline defined as YAML workflow files version-controlled inside the SupportHub repository.	No separate Jenkins server to provision or maintain. Workflow files live alongside code. Cloud-native, free tier sufficient for this project.
4	**Terraform Local Provider**	Terraform uses hashicorp/local and hashicorp/null providers — no cloud account required. Scripts provision the environment locally via shell.	Maximally accessible for evaluation. Any evaluator runs terraform apply on any Linux machine and gets the full SupportHub platform running. Directly fulfills DO-15 IaC requirement.
5	**Nginx as Single Entry Point + Landing Page**	One Nginx container receives all traffic on port 80, serves the SupportHub landing page at /, and proxies /triage/* and /quiz/* with rewrite rules.	Single public port — backends never directly exposed. Path-based rewrites mean each app receives its original API paths without modification. Landing page is served at zero additional cost.
6	**Kubernetes Orchestration**	K8s Deployments manage lifecycle, replication, livenessProbes, and rolling updates for all services with defined resource limits.	Self-healing: crashed containers restart automatically. Rolling updates enable zero-downtime deploys. Resource limits prevent memory leaks from starving the host.
7	**Secrets via .env + GitHub Secrets**	API keys, Firebase credentials, and JWT secrets stored in .env files. Never hardcoded in Dockerfiles or app source code.	Hardcoded secrets in Docker images is a critical security vulnerability. .env files are gitignored. CI/CD uses encrypted GitHub repository Secrets for DOCKERHUB_USERNAME and DOCKERHUB_TOKEN.

2.5 API Catalogue

BBase URL (AWS): http://13.206.119.123:8080 **| Network:** supporthub-network **| Auth:** Authorization: Bearer {JWT} | Content-Type: application/json

Nginx Routing Configuration (nginx/nginx.conf): Nginx defines **upstream blocks** for each container by Docker service name, then uses **location blocks with rewrite rules** to strip the service prefix before proxying. This allows each app to continue using its original internal API paths unchanged.

Location Block	Proxy Pass Target	Rewrite / Purpose
/	root /usr/share/nginx/html (static)	Serves portal/index.html — SupportHub landing page
/triage/api/	http://triage_backend (upstream: triage-backend:8000)	Rewrite: /triage/api/(.*) → /api/$1. TriageAgent backend receives /api/* as normal.
/triage/	http://triage_frontend (upstream: triage-frontend:3000)	Rewrite: /triage/(.*) → /$1. TriageAgent React app served from root.
/quiz/api/	http://quiz_backend (upstream: quiz-backend:8001)	Rewrite: /quiz/api/(.*) → /api/$1. QuizBot backend receives /api/* as normal.
/quiz/	http://quiz_frontend (upstream: quiz-frontend:3001)	Rewrite: /quiz/(.*) → /$1. QuizBot React app served from root.

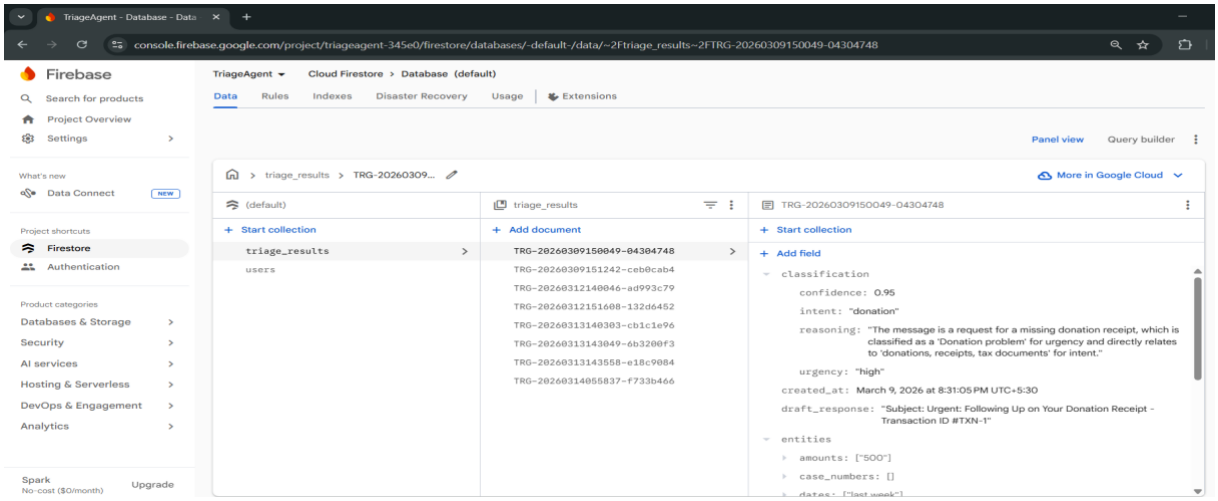
3. Data Design

3.1 Data Model

SupportHub uses three distinct data stores across its two microservices. Firebase Firestore (NoSQL cloud database) is shared by both for authentication and primary data. ChromaDB (vector database) and SQLite are used exclusively by QuizBot for semantic search and quiz result persistence.

3.1.1 Firebase Database Schema _TriageAgent

TriageAgent uses exactly two root Firestore collections: triage_results and users. The entire state of a triage result is captured in a single document — including embedded classification, entities, and routing objects stored as Firestore MAPs.



3.1.2 QuizBot Data Stores

Store	Type	Purpose	Persistence
Firebase Firestore	NoSQL Cloud	User auth profiles, quiz history, performance analytics	Cloud (permanent)
ChromaDB	Vector DB	Stores embeddings of donor emails for semantic similarity search during quiz generation	Docker volume ./data/ (persists across restarts)
SQLite (quizbot.db)	Relational DB	Quiz sessions, question results, scores, user progress tracking	Docker volume ./quizbot.db (persists across restarts)

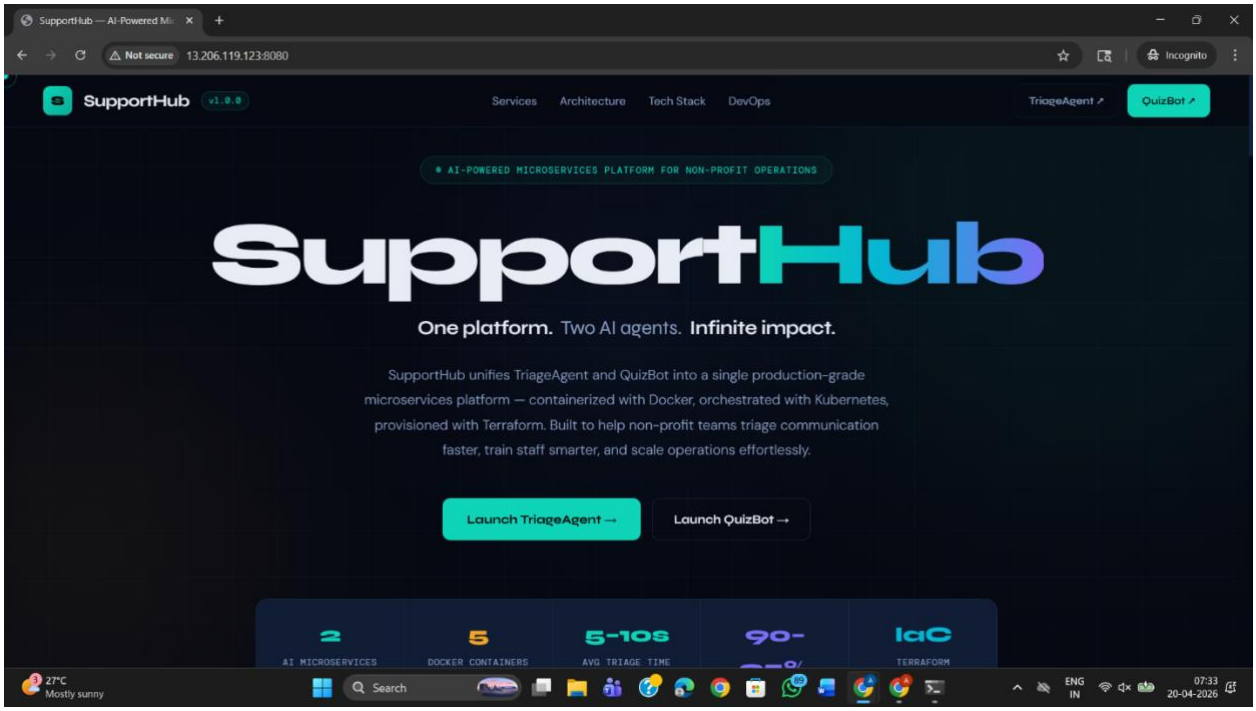
3.2 Data Access Mechanism

All Firestore access in both microservices is mediated exclusively through service classes (FirebaseService in TriageAgent, AuthService in QuizBot). No route handler or agent interacts with Firestore directly. This centralization ensures consistent error handling, testability, and the ability to replace the database layer in isolation.

4. Interfaces

4.1 User Interfaces (UI)

Page / App	URL	Purpose
SupportHub Portal	http://13.206.119.123:8080/	Landing page — links to both microservices, platform overview
TriageAgent Login	http://13.206.119.123:8080/triage/login	Email + password login via Firebase Auth
TriageAgent Triage	http://13.206.119.123: 8080/triage/triage	Main AI triage interface — message input + 3-tab result display
TriageAgent Dashboard	http://13.206.119.123:8080/triage/dashboard	Paginated message list with urgency/status filters
TriageAgent Analytics	http://13.206.119.123:8080/triage/analytics	PieChart (urgency), BarChart (intent), StatsCards, KPIs
QuizBot Login	http://13.206.119.123:8080/quiz/login	Email + password login via Firebase Auth
QuizBot Dashboard	http://13.206.119.123:8080/quiz/dashboard	Upload email, generate quiz, track progress
API Docs (TriageAgent)	http://13.206.119.123:8080/triage/api/docs	FastAPI auto-generated Swagger/OpenAPI documentation
API Docs (QuizBot)	http://13.206.119.123:8080/quiz/api/docs	FastAPI auto-generated Swagger/OpenAPI documentation



4.2 Internal Module Interfaces

All frontend-to-backend communication goes through a single Axios instance configured with the backend base URL. A request interceptor automatically attaches the JWT Bearer token to every outgoing request from the Zustand auth store. A response interceptor catches 401 responses, clears the Zustand state, and redirects to /login.

Agent / Service	Input	Output
ClassifierAgent → LLMService	message: str	Classification {urgency: UrgencyLevel, intent: IntentCategory, confidence: float, reasoning: str}
NERAgent → NERService	message: str	EntityExtraction {persons[], dates[], amounts[], emails[], receipt_ids[], case_numbers[]}
ResponseAgent → LLMService	message, classification, entities	draft_response: str (Subject line + email body)
RouterAgent (pure logic)	classification: Classification	RoutingDecision {department: DepartmentType, priority: int(1–4)
Orchestrator → FirebaseService	TriageResult dict	triage_id: str (write confirmation)
AnalyticsService → FirebaseService	days: int	stats dict {by_urgency, by_intent, approval_rate, trending_intents}

4.3 External Service Interfaces

External System	Protocol	Library	Auth	Key Operations
Google Gemini 2.5 Flash	HTTPS REST	google-genai	GEMINI_API_KEY (.env)	generate_content() — classify, NER, response generation
Firebase Auth	Firebase Admin SDK	firebase-admin	Service account JSON	create_user(), verify_id_token()
Firebase Firestore	Firebase Admin SDK	firebase-admin	Same service account	set(), get(), update(), where().get()
spaCy NLP	In-process (no network)	spacy	None	nlp(text) → doc.ents — loaded once at startup
Docker Hub Registry	HTTPS	docker CLI / buildx	DOCKERHUB_TOKEN (GitHub Secret)	docker pull / push all 5 images
AWS EC2	HTTPS REST (Terraform)	hashicorp/aws provider	AWS_ACCESS_KEY + SECRET	Create VPC, subnet, SG, EC2, Elastic IP

5. Infrastructure & Deployment

5.1 Docker Architecture

Quizbot-backend Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    curl \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

ENV PIP_DEFAULT_TIMEOUT=300

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8001

CMD ["sh", "-c", "python init_db.py && uvicorn app.main:app --host 0.0.0.0 --port 8001"]
```

Quizbot-frontend Dockerfile

```
WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 3001

CMD ["nginx", "-g", "daemon off;"]
```

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    gcc \
    g++ \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

RUN python -m spacy download en_core_web_sm

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Triage-backend Dockerfile

```
WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 3000

CMD ["nginx", "-g", "daemon off;"]
```

Triage-backend Dockerfile

Nginx – Dockerfile

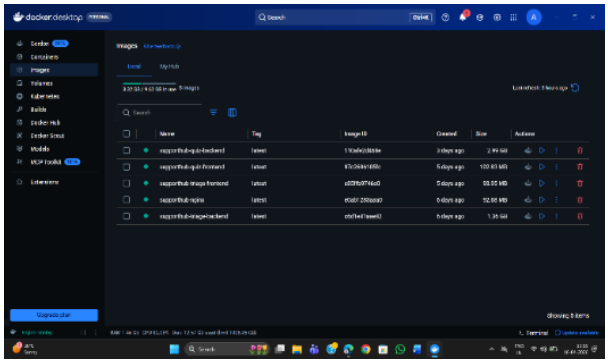
```
FROM nginx:alpine

COPY nginx/nginx.conf /etc/nginx/nginx.conf
COPY portal/index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

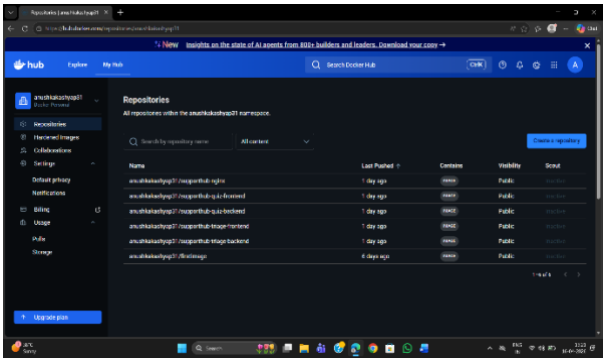
Docker Compose – Service Orchestration

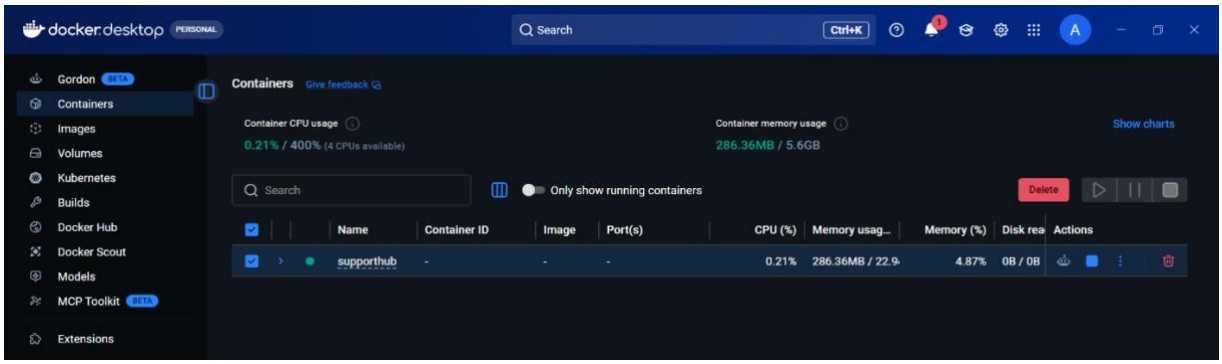
```
services:
  nginx:
    build:
      context: .
      dockerfile: nginx/Dockerfile
    container_name: supporthub-nginx
    ports:
      - "80:80"
    depends_on:
      - triage-frontend
      - triage-backend
      - quiz-frontend
      - quiz-backend
    restart: unless-stopped
    networks:
      - supporthub-network

networks:
  supporthub-network:
    driver: bridge
```

2.5.2 Nginx Configuartion – API Gateway

```
http {
    server {
        listen 80;

        location /triage/api/ {
            rewrite ^/triage/api/(.*) /api/$1 break;
            proxy_pass http://triage_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /triage/ {
            rewrite ^/triage/(.*) /$1 break;
            proxy_pass http://triage_frontend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /quiz/api/ {
            rewrite ^/quiz/api/(.*) /api/$1 break;
            proxy_pass http://quiz_backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /quiz/ {
            rewrite ^/quiz/(.*) /$1 break;
            proxy_pass http://quiz_frontend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location / {
            root /usr/share/nginx/html;
            index index.html;
            try_files $uri $uri/ /index.html;
        }
```

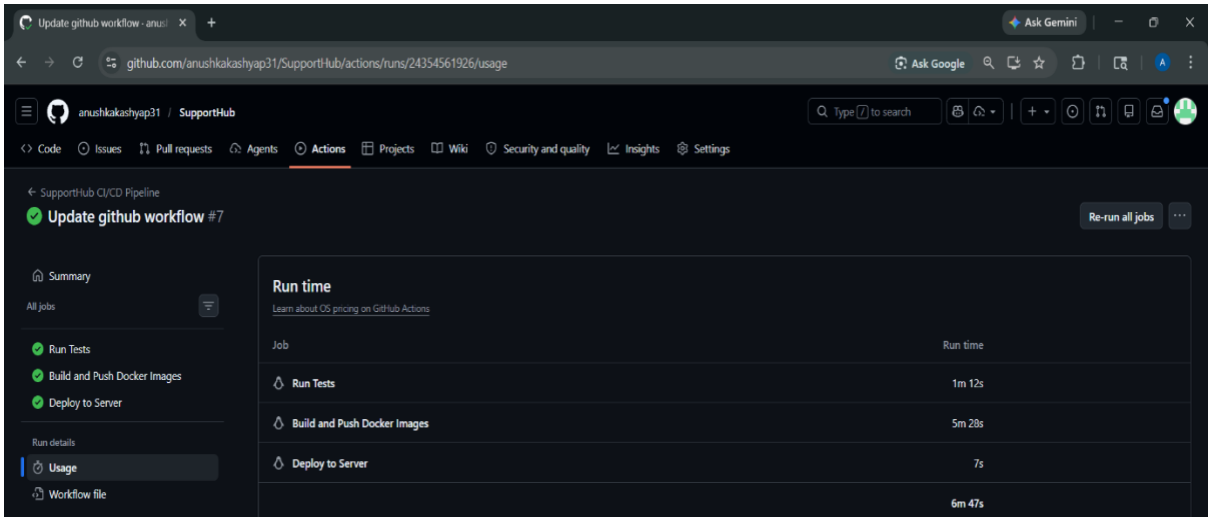






5.2 CI/CD Pipeline — GitHub Actions

File: .github/workflows/ci-cd.yml | Status: ALL 3 JOBS PASSING ✅ | Run #7 — 6m 47s
The SupportHub CI/CD pipeline in **.github/workflows/ci-cd.yml** uses GitHub Actions with 3 sequential jobs. Every push to the main branch triggers an automated workflow that **tests** (import validation), **builds and push** (Docker images to DockerHub), and **deploy** to server (compose restart). Jobs 2 and 3 only run on pushes to main.



5.3 Terraform IaC

Terraform automates the complete AWS infrastructure provisioning. A single terraform apply command creates the entire SupportHub cloud environment — from networking to the running server — with zero manual steps.

File	Purpose
terraform/main.tf	AWS provider (ap-south-1), VPC (10.0.0.0/16), subnet, internet gateway, route table, security group (ports: 22/80/30080/8000/8001/3000/3001), key pair, EC2 t3.small (Ubuntu 24.04, 20GB gp2), Elastic IP, K3s via user_data
terraform/variables.tf	project_name, environment, aws_region (ap-south-1), instance_type (t3.small), ami_id (ami-05d2839d4f73aafb), public_key_path, all service ports
terraform/outputs.tf	server_public_ip, supporthub_url, triage_url, quiz_url, 15ubernetes_url (:30080), ssh_command
terraform/scripts/install.sh	Auto-installs Docker CE, Docker Compose, Terraform 1.14, kubectl on fresh Ubuntu. Idempotent — checks if already installed before installing.
Terraform/scripts/deploy.sh	Verifies .env exists, builds images from docker-compose.yml, runs docker compose up -d

Terraform Deployment Commands

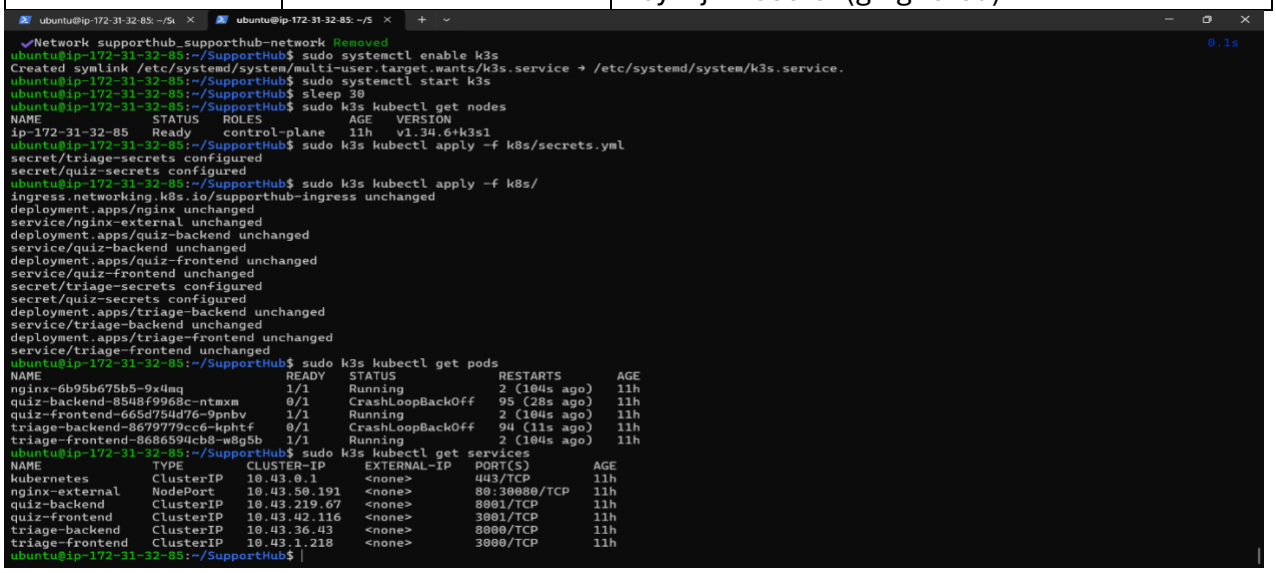
Command	Action
terraform init	Downloads AWS provider plugin (~5.0)
terraform plan	Preview all resources to be created
terraform apply	Creates entire AWS infrastructure (type 'yes')
terraform output	Print EC2 public IP and SupportHub URLs
terraform destroy	Terminates all resources, stops billing

5.4 Kubernetes Orchestration

SupportHub includes complete Kubernetes manifests for all 5 services, enabling production-grade orchestration with auto-restart, health probes, horizontal scaling, and resource management. K3s (lightweight Kubernetes) is used on AWS EC2 for single-server deployment.

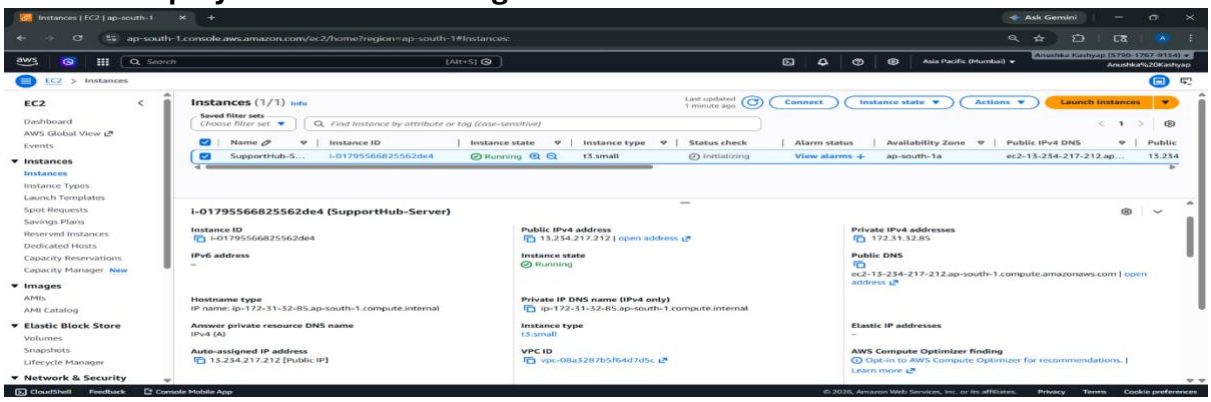
File	Kind	Key Spec
triage-backend-deployment.yml	Deployment	image: anushkakashyap31/supporthub-triage-backend:latest | livenessProbe GET /health :8000 | requests: 256Mi/250m | limits: 512Mi/500m
triage-backend-service.yml	Service (ClusterIP)	port: 8000 → targetPort: 8000 | Internal cluster DNS only
triage-frontend-deployment.yml	Deployment	image: anushkakashyap31/supporthub-triage-frontend:latest | port: 3000
quizbot-backend-deployment.yml	Deployment	image: anushkakashyap31/supporthub-quiz-backend:latest | requests: 512Mi | livenessProbe /health :8001
quizbot-frontend-deployment.yml	Deployment	image: anushkakashyap31/supporthub-quiz-frontend:latest | port: 3001
nginx-deployment.yml	Deployment	image: anushkakashyap31/supporthub-nginx:latest | port: 80

nginx-service.yml	Service (NodePort)	port: 80 → nodePort: 30080 | External access via EC2 IP:30080
ingress.yml	Ingress	Path-based routing: /triage → triage-frontend, /quiz → quiz-frontend | rewrite-target: /
secrets.yml	Secret (Opaque)	triage-secrets and quiz-secrets — gemini-api-key + jwt-secret (gitignored)



```
✓Network supporthub_supporthub-network Removed
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo systemctl enable k3s
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/systemd/system/k3s.service.
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo systemctl start k3s
ubuntu@ip-172-31-32-85:~/SupportHub$ sleep 30
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo k3s kubectl get nodes
NAME              STATUS   ROLES           AGE   VERSION
ip-172-31-32-85   Ready    control-plane   11h   v1.34.6+k3s1
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo k3s kubectl apply -f k8s/secrets.yml
secret/triage-secrets configured
secret/quiz-secrets configured
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo k3s kubectl apply -f k8s/
ingress.networking.k8s.io/supporthub-ingress unchanged
deployment.apps/nginx unchanged
service/nginx-external unchanged
deployment.apps/quiz-backend unchanged
service/quiz-backend unchanged
deployment.apps/quiz-frontend unchanged
service/quiz-frontend unchanged
secret/triage-secrets configured
secret/quiz-secrets configured
deployment.apps/triage-backend unchanged
service/triage-backend unchanged
deployment.apps/triage-frontend unchanged
service/triage-frontend unchanged
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo k3s kubectl get pods
NAME                              READY   STATUS             RESTARTS       AGE
nginx-6b95b675b5-9x4mq            1/1     Running            2 (104s ago)   11h
quiz-backend-8548f9968c-ntmxm     0/1     CrashLoopBackOff   95 (28s ago)   11h
quiz-frontend-665d754d76-9pnbv    1/1     Running            2 (104s ago)   11h
triage-backend-8679779cc6-kphtf   0/1     CrashLoopBackOff   94 (11s ago)   11h
triage-frontend-8686594cb8-w8g5b  1/1     Running            2 (104s ago)   11h
ubuntu@ip-172-31-32-85:~/SupportHub$ sudo k3s kubectl get services
NAME             TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes       ClusterIP   10.43.0.1      <none>        443/TCP        11h
nginx-external   NodePort    10.43.50.191   <none>        80:30080/TCP   11h
quiz-backend     ClusterIP   10.43.219.67   <none>        8001/TCP       11h
quiz-frontend    ClusterIP   10.43.42.116   <none>        3001/TCP       11h
triage-backend   ClusterIP   10.43.36.43    <none>        8000/TCP       11h
triage-frontend  ClusterIP   10.43.1.218    <none>        3000/TCP       11h
ubuntu@ip-172-31-32-85:~/SupportHub$
```

5.5 AWS Deployment — Live Configuration



Component	Configuration
EC2 Instance	t3.small | Ubuntu 24.04 LTS | ap-south-1 | 20GB gp2 | Public IP: 13.206.119.123:8080
Elastic IP	Static public IP — survives instance stop/start
Security Group	Inbound: 22 (SSH), 80 (HTTP), 30080 (K8s NodePort), 8000, 8001, 3000, 3001 | Outbound: all
Docker Images	All 5 pulled from Docker Hub: anushkakashyap31/supporthub-* | 5 containers running (verified)
Platform URL	http://13.206.119.123:8080 → SupportHub landing page
TriageAgent URL	http://13.206.119.123:8080/triage/ → TriageAgent application
QuizBot URL	http://13.206.119.123:8080/quiz/ → QuizBot application
Kubernetes	K3s installed via Terraform user_data | kubectl apply -f k8s/ → 5 pods + NodePort :30080

6. State and Session Management

SupportHub manages all application state through Zustand stores on the frontend. Authentication uses a dual-token model — Firebase Auth verifies user identity and a custom JWT handles API access control. The JWT is stored in localStorage via Zustand persist middleware and attached to every API request automatically by an Axios interceptor.

Store	State Managed	Persistence	Cleared When
useAuthStore	token, user (uid, email, role), isAuthenticated	localStorage via Zustand persist	logout() called or HTTP 401 received
useTriageStore	currentResult, history[], isProcessing, isLoading	Session memory only (not persisted)	Page refresh or clearCurrent() action
useMessageStore	messages[], filters, isLoading	Session memory only	Page refresh or reset() action
JWT Token	HS256 signed (uid, email, role, exp)	localStorage via Zustand	Expires after 30 min or logout
QuizBot useQuizStore	currentQuiz, quizResults, progress	Session memory only	Page refresh

Session Lifecycle: App loads → checkAuth() reads localStorage → if JWT valid, isAuthenticated=true and protected routes open → if JWT invalid/expired → redirect to /login. On logout: clear Zustand state + localStorage → redirect to /login.

7. Caching

SupportHub v1.0 uses lightweight built-in caching only — no Redis, no CDN, no dedicated cache layer. Caching is applied at three points: the browser (JWT + state via Zustand persist), the backend startup (spaCy model), and the frontend build output (Vite asset hashing).

What is Cached	Where	Cache Key	Invalidated When
JWT access token	localStorage	zustand-auth	logout() called or 30 min expiry
User profile (uid, email, role)	localStorage	zustand-auth	logout() called
Current TriageResult	Zustand (memory)	useTriageStore	Page refresh or clearCurrent()
Message history (last 10)	Zustand (memory)	useTriageStore	Page refresh
spaCy NLP model (en_core_web_sm)	Backend memory (singleton)	NERService class var	Container restart — never expires during runtime
Frontend build assets	Browser cache	Content hash in filename	Code changes → new hash → forced re-download

8. Non-Functional Requirements

8.1 Security Aspects

#	Security Aspect	Description
1	Authentication & Authorization	All API endpoints except /register and /login are protected by JWT Bearer token. Tokens signed with HS256, SECRET_KEY from .env, expire after 30 minutes. Firebase handles all password storage — backend never stores raw passwords.
2	Transport Security	All frontend ↔ backend communication over HTTP locally, HTTPS in production. All external API calls (Gemini API, Firebase Admin SDK) use HTTPS exclusively. Nginx terminates connections.
5	Secret Management	GEMINI_API_KEY, JWT_SECRET, FIREBASE_CREDENTIALS_PATH stored in .env, listed in .gitignore, never committed. GitHub Secrets store DOCKERHUB USERNAME and TOKEN for CI/CD. Firebase service account JSON uploaded to EC2 via scp.
6	Firestore Access Control	All Firestore reads/writes go exclusively through FirebaseService via Firebase Admin SDK (server-side only). Direct client-side Firestore access blocked by Firestore Security Rules.
5	Container Security	Docker images built from official minimal base images (python:3.11-slim, node:20-alpine, nginx:alpine). No secrets baked into images. Images scanned via Docker Hub.
7	Kubernetes Secrets	API keys and JWT secrets stored in Kubernetes Opaque Secrets (k8s/secrets.yml). Referenced by secretKeyRef in pod specs — never exposed as environment variables in plain text.
8	AWS Security Group	EC2 security group allows only: port 22 (SSH), port 80 (HTTP), port 30080 (K8s NodePort), ports 8000/8001 (APIs), ports 3000/3001 (frontends). All other inbound blocked.

Network Security

- All services communicate internally via Docker bridge network — no internal ports exposed externally
- AWS Security Group restricts inbound traffic to only required ports (22, 80, 8000, 8001, 3000, 3001)
- SSH access restricted to key-pair authentication (supporthub-key.pem)
- firebase-credentials.json excluded from Git via .gitignore
- Environment variables never committed to source code

Performance Aspects

Operation	Target	Notes
Full triage pipeline	5–10 sec	Classify + NER + Response + Route + Firestore save (all 4 agents)
QuizBot quiz generation	**8-15 seconds**	Gemini generates questions from email content via ChromaDB context
API validation only	< 50 ms	Pydantic V2 + JWT verify, no AI call
Auth login	< 500 ms	Firebase verify + JWT issue
Docker build (first time)	15–25 min	Downloads Python packages, PyTorch CPU, spaCy model
Docker compose up	< 2 min	Subsequent starts using cached images
CI/CD pipeline	**~7 minutes**	Test: 1m 12s | Build-and-Push: 5m 28s | Deploy: 7s

9. Reference

#	Reference	URL / Source
1	Docker Official Documentation	https://docs.docker.com/
2	Docker Compose Documentation	https://docs.docker.com/compose/
3	GitHub Actions Documentation	https://docs.github.com/en/actions
4	Terraform AWS Provider	https://registry.terraform.io/providers/hashicorp/aws/latest/docs
5	Kubernetes Documentation	https://kubernetes.io/docs/
6	K3s Lightweight Kubernetes	https://k3s.io/
7	Nginx Documentation	https://nginx.org/en/docs/
8	FastAPI Official Documentation	https://fastapi.tiangolo.com/
9	LangChain Documentation	https://python.langchain.com/docs/
10	Google Gemini API	https://ai.google.dev/gemini-api/docs
11	spaCy NLP Documentation	https://spacy.io/usage
12	Firebase Firestore	https://firebase.google.com/docs/firestore
13	ChromaDB Vector Database	https://docs.trychroma.com/
14	Sentence-Transformers	https://sbert.net/
15	React 19 Documentation	https://react.dev/
16	Vite Build Tool	https://vitejs.dev/
17	Zustand State Management	https://zustand.docs.pmnd.rs/
18	SupportHub GitHub Repository	https://github.com/anushkakashyap31/SupportHub